# Spring Core & Maven — Lab Assignment

Project: LibraryManagement | Framework: Spring Core, Spring AOP, Spring Boot | Build Tool: Maven

This document contains complete, working solutions for all nine exercises, building incrementally towards a full Spring Boot–based Library Management application. All package names follow `com.library.*` as specified in the assignment brief.

---

## Exercise 1: Configuring a Basic Spring Application

> *Scenario: Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.*

### 1. pom.xml — Spring Core dependencies

**pom.xml**

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
        <spring.version>5.3.30</spring.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>${spring.version}</version>
        </dependency>
    </dependencies>
</project>
```

### 2. applicationContext.xml

**src/main/resources/applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository"/>

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

</beans>
```

### 3. Service and Repository classes

**src/main/java/com/library/repository/BookRepository.java**

```
package com.library.repository;

public class BookRepository {
    public void save(String bookTitle) {
        System.out.println("Book saved to repository: " + bookTitle);
    }
}
```

`src/main/java/com/library/service/BookService.java`

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook(String title) {
        bookRepository.save(title);
    }
}
```

### 4. Main class to load context

`src/main/java/com/library/LibraryManagementApplication.java`

```
package com.library;

import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService bookService = (BookService) context.getBean("bookService");
        bookService.addBook("Effective Java");
    }
}
```

**Expected console output:** `Book saved to repository: Effective Java`

---

## Exercise 2: Implementing Dependency Injection

> *Scenario: Manage the dependencies between BookService and BookRepository using Spring's IoC and DI.*

### 1. Wire BookRepository into BookService (setter injection)

`src/main/resources/applicationContext.xml (updated)`

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository"/>

    <bean id="bookService" class="com.library.service.BookService">
        <!-- Setter injection: Spring calls setBookRepository() automatically -->
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

</beans>
```

### 2. BookService setter (already present in Exercise 1)

The `setBookRepository()` method defined earlier already satisfies this requirement — Spring's IoC container invokes it at bean-creation time to inject the collaborator.

### 3. Test the configuration

```
Running LibraryManagementApplication...
Output:
Book saved to repository: Effective Java
```

The absence of a `NullPointerException` confirms that `bookRepository` was successfully injected into `bookService` by the container before `addBook()` was invoked.

---

## Exercise 3: Implementing Logging with Spring AOP

> *Scenario: Add logging capabilities to track method execution times.*

### 1. Add Spring AOP dependency

**pom.xml (additions)**

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
    <version>1.9.20</version>
</dependency>
```

### 2. LoggingAspect class

**src/main/java/com/library/aspect/LoggingAspect.java**

```
package com.library.aspect;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;

@Aspect
public class LoggingAspect {

    @Around("execution(* com.library.service.*.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long elapsed = System.currentTimeMillis() - start;
        System.out.println(joinPoint.getSignature() + " executed in " + elapsed + " ms");
        return result;
    }
}
```

### 3. Enable AspectJ support in XML

**src/main/resources/applicationContext.xml (additions)**

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/aop
           http://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository"/>

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

    <bean id="loggingAspect" class="com.library.aspect.LoggingAspect"/>

    <aop:aspectj-autoproxy/>
```

```
</beans>
```

### 4. Expected console output

```
Book saved to repository: Effective Java
execution(void com.library.service.BookService.addBook(String)) executed in 2 ms
```

---

## Exercise 4: Creating and Configuring a Maven Project

> *Scenario: Set up a new Maven project and add all required Spring dependencies.*

### 1. Create the Maven project

```
mvn archetype:generate -DgroupId=com.library -DartifactId=LibraryManagement \
    -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

### 2 & 3. Full pom.xml with dependencies and compiler plugin

`pom.xml`

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
        <spring.version>5.3.30</spring.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>${spring.version}</version>
        </dependency>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-aop</artifactId>
            <version>${spring.version}</version>
        </dependency>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-webmvc</artifactId>
            <version>${spring.version}</version>
        </dependency>
        <dependency>
            <groupId>org.aspectj</groupId>
            <artifactId>aspectjweaver</artifactId>
            <version>1.9.20</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
                <configuration>
                    <source>1.8</source>
                    <target>1.8</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

| Dependency | Purpose |
| --- | --- |
| | |

| | |
|---|---|
| spring-context | Core IoC container (ApplicationContext, bean wiring) |
| spring-aop | Aspect-oriented programming support |
| spring-webmvc | MVC / REST controller support |
| aspectjweaver | Enables @Aspect / @Around annotations |

---

## Exercise 5: Configuring the Spring IoC Container

> *Scenario: The application requires a central configuration for beans and dependencies.*

This exercise reinforces the setup completed in Exercises 1–2. The consolidated `applicationContext.xml` and `BookService` setter method are shown below for completeness.

**src/main/resources/applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository"/>

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

</beans>
```

**src/main/java/com/library/service/BookService.java**

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook(String title) {
        bookRepository.save(title);
    }
}
```

Running `LibraryManagementApplication` confirms the container starts correctly and both beans are wired and functional.

---

## Exercise 6: Configuring Beans with Annotations

> *Scenario: Simplify bean configuration using annotations instead of explicit XML bean definitions.*

### 1. Enable component scanning

**src/main/resources/applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd">
```

```
    <context:component-scan base-package="com.library"/>

</beans>
```

### 2. Annotate the classes

`src/main/java/com/library/repository/BookRepository.java`

```
package com.library.repository;

import org.springframework.stereotype.Repository;

@Repository
public class BookRepository {
    public void save(String bookTitle) {
        System.out.println("Book saved to repository: " + bookTitle);
    }
}
```

`src/main/java/com/library/service/BookService.java`

```
package com.library.service;

import com.library.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class BookService {

    private BookRepository bookRepository;

    @Autowired
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook(String title) {
        bookRepository.save(title);
    }
}
```

### 3. Test the configuration

Running `LibraryManagementApplication` produces the same output as before, confirming that Spring detected the `@Service` and `@Repository` annotated classes via component scanning and autowired the dependency without any explicit `<bean>` declarations.

---

## Exercise 7: Implementing Constructor and Setter Injection

> *Scenario: Support both constructor and setter injection for better control over bean initialization.*

### 1 & 2. BookService with both injection styles

`src/main/java/com/library/service/BookService.java`

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    // Constructor injection
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    // Setter injection (allows re-wiring / optional override)
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
```

```
    }

    public void addBook(String title) {
        bookRepository.save(title);
    }
}
```

### 3. XML configuration for constructor + setter injection

`src/main/resources/applicationContext.xml`

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository"/>

    <bean id="bookService" class="com.library.service.BookService">
        <!-- Constructor injection -->
        <constructor-arg ref="bookRepository"/>
        <!-- Setter injection (re-applies / confirms the same dependency) -->
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

</beans>
```

### 4. Test

Running the application confirms both injection paths succeed — the constructor initializes the bean and the setter can subsequently override the reference, with no `NullPointerException` at any stage.

---

## Exercise 8: Implementing Basic AOP with Spring

> *Scenario: Separate cross-cutting concerns like logging and transaction management using AOP.*

### 1 & 2. Aspect with Before/After advice

`src/main/java/com/library/aspect/LoggingAspect.java`

```
package com.library.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;

@Aspect
public class LoggingAspect {

    @Before("execution(* com.library.service.*.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("[BEFORE] Entering method: " + joinPoint.getSignature().getName());
    }

    @After("execution(* com.library.service.*.*(..))")
    public void logAfter(JoinPoint joinPoint) {
        System.out.println("[AFTER] Exiting method: " + joinPoint.getSignature().getName());
    }
}
```

### 3. Register the aspect and enable auto-proxying

`src/main/resources/applicationContext.xml`

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
```

```
           http://www.springframework.org/schema/aop
           http://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository"/>

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

    <bean id="loggingAspect" class="com.library.aspect.LoggingAspect"/>

    <aop:aspectj-autoproxy/>

</beans>
```

### 4. Expected console output

```
[BEFORE] Entering method: addBook
Book saved to repository: Effective Java
[AFTER] Exiting method: addBook
```

---

## Exercise 9: Creating a Spring Boot Application

> *Scenario: Create a Spring Boot application for the library management system to simplify configuration and deployment.*

### 1. Spring Boot starter pom.xml

`pom.xml`

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.7.18</version>
    </parent>

    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <java.version>1.8</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

### 2. application.properties

`src/main/resources/application.properties`

```
spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

## 3. Entity and Repository

`src/main/java/com/library/entity/Book.java`

```
package com.library.entity;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;

    public Book() {}

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }

    public Long getId() { return id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
}
```

`src/main/java/com/library/repository/BookRepository.java`

```
package com.library.repository;

import com.library.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

## 4. REST Controller

`src/main/java/com/library/controller/BookController.java`

```
package com.library.controller;

import com.library.entity.Book;
import com.library.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    @Autowired
    private BookRepository bookRepository;

    @GetMapping
```

```
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    @GetMapping("/{id}")
    public Book getBookById(@PathVariable Long id) {
        return bookRepository.findById(id).orElse(null);
    }

    @PostMapping
    public Book createBook(@RequestBody Book book) {
        return bookRepository.save(book);
    }

    @PutMapping("/{id}")
    public Book updateBook(@PathVariable Long id, @RequestBody Book updatedBook) {
        return bookRepository.findById(id).map(book -> {
            book.setTitle(updatedBook.getTitle());
            book.setAuthor(updatedBook.getAuthor());
            return bookRepository.save(book);
        }).orElse(null);
    }

    @DeleteMapping("/{id}")
    public void deleteBook(@PathVariable Long id) {
        bookRepository.deleteById(id);
    }
}
```

## 5. Main application class

`src/main/java/com/library/LibraryManagementApplication.java`

```
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
    }
}
```

## 6. Testing the REST endpoints

```
curl -X POST http://localhost:8080/api/books \
     -H "Content-Type: application/json" \
     -d '{"title":"Effective Java","author":"Joshua Bloch"}'

curl http://localhost:8080/api/books

curl http://localhost:8080/api/books/1

curl -X PUT http://localhost:8080/api/books/1 \
     -H "Content-Type: application/json" \
     -d '{"title":"Effective Java (3rd Ed.)","author":"Joshua Bloch"}'

curl -X DELETE http://localhost:8080/api/books/1
```

Running `mvn spring-boot:run` starts an embedded Tomcat server on port 8080; the H2 in-memory database persists `Book` records for the lifetime of the run, and all five CRUD endpoints respond as shown above.

---